Assessment: Spring Boot, Hibernate, JPA & Dependency Injection

Total Marks: 50

Duration: 90 Minutes

Section A – Multiple Choice Questions (Code Based) (10 × 2 = 20 Marks)

Choose the correct answer.

1. Hibernate Entity Mapping

```
@Entity
@Table(name="employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

What does @Entity indicate?
A. The class is a Spring service
B. The class will be mapped to a database table
C. The class is used for REST API
D. The class stores configuration

2. Dependency Injection

```
@Service
public class PaymentService {

    private PaymentGateway gateway;

    @Autowired
    public PaymentService(PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```

What type of dependency injection is used?
A. Field Injection
B. Constructor Injection
C. Setter Injection
D. Static Injection

3. JPA Repository

```
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

What functionality does this interface provide automatically?
A. Database connection pooling
B. CRUD operations on Student entity
C. REST API generation
D. Transaction management

4. REST Controller

```
@RestController
@RequestMapping("/students")
public class StudentController {

  @GetMapping
  public List<Student> getStudents(){
    return service.getAllStudents();
  }
}
```

Which annotation makes the class a REST API controller?
A. @Controller
B. @RestController
C. @Component
D. @Service

5. Spring Data Query Method

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

  List<Employee> findByDepartment(String dept);
}
```

What does this method do?
A. Generates SQL manually
B. Automatically creates a query based on method name

C. Requires Hibernate XML configuration
D. Calls stored procedure

6. Entity Relationship

```
@Entity
public class Order {

    @ManyToOne
    private Customer customer;
}
```

What does @ManyToOne represent?

A. One order has many customers
B. Many orders can belong to one customer
C. One customer belongs to many orders
D. Many customers belong to many orders

7. Transaction Management

```
@Service
public class AccountService {

    @Transactional
    public void transferMoney(){
        // database operations
    }
}
```

What is the purpose of @Transactional?

A. Start REST service
B. Manage database transaction automatically
C. Configure dependency injection
D. Create database table

8. Fetching Data Using Repository

```
@GetMapping("/{id}")
public ? getStudent(@PathVariable Long id){
    return studentRepository.findById(id).orElse(null);
}
```

What does findById() return?

A. List<Student>
B. Student
C. Optional<Student>
D. Map<Student>

9. Hibernate Configuration

```
spring.jpa.hibernate.ddl-auto=update
```

What does this configuration do?

A. Deletes database every restart
B. Automatically updates schema based on entities
C. Disables Hibernate
D. Enables REST endpoints

10. Component Scanning

```
@Component
public class EmailService {

}
```

What is the role of **@Component**?

A. Marks class for Spring Bean creation
B. Creates database table
C. Creates REST API
D. Enables transaction

Section B – Coding Questions (3 × 10 = 30 Marks)

Question 11 (10 Marks)

Create a JPA Entity class **Product** with the following fields:
id : Long
name : String
price : Double
category : String

Requirements:
1. Use appropriate JPA annotations
2. Map entity to table 'products'
3. Use auto-generated primary key
4. Include getters and setters

Question 12 (10 Marks)

Write a Spring Data JPA Repository interface for Product entity.

Requirements:
1. Extend JpaRepository
2. Create a method to find products by category
3. Create a method to find products cheaper than a given price

Example methods:
findByCategory(String category)
findByPriceLessThan(Double price)

Question 13 (10 Marks)

Create a REST Controller ProductController.

Requirements:
1. Inject ProductRepository
2. Create the following endpoints

GET /products -> Get all products
GET /products/{id} -> Get product by id
POST /products -> Add new product